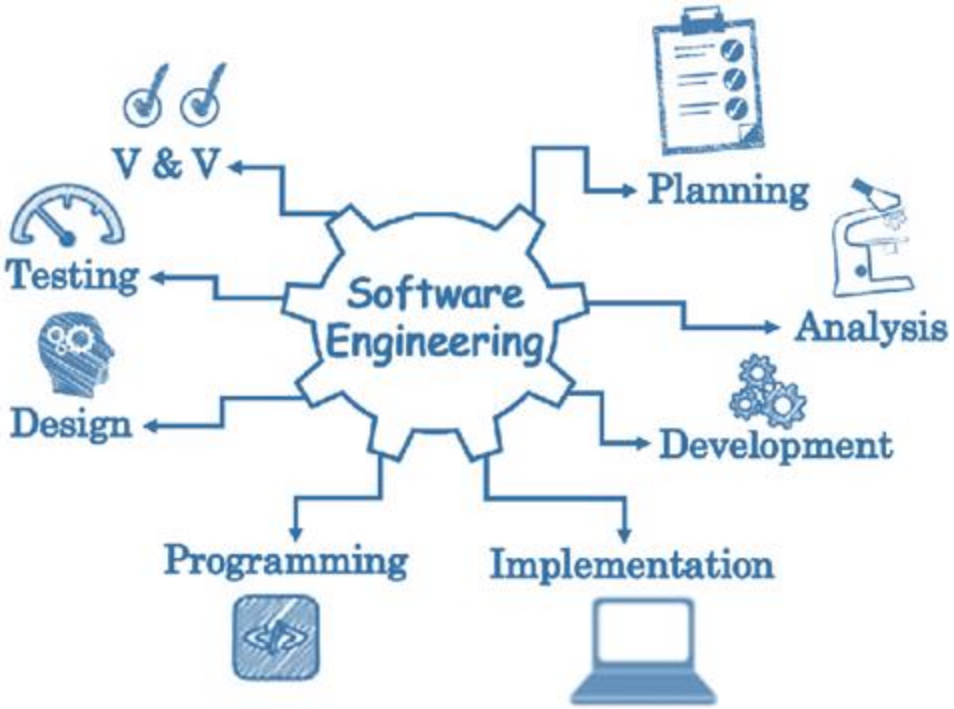
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Software Process Models and Project

4.1 OOAD Principles

- Encapsulation
- Abstraction
- Inheritance
- Polymorphism

4.2 Analysis Object Models and Dynamic Models

- Entity, Boundary, and Control Objects
- Generalization and Specialization

4.6 Case Study

- Applying OOAD and UML to a Real-World Application

4.3 UML Diagrams

- Use Case Diagrams
- Class Diagrams
- Sequence Diagrams
- Activity Diagrams
- State Machine Diagrams

4.4 Modeling Interactions and Object Lifecycles

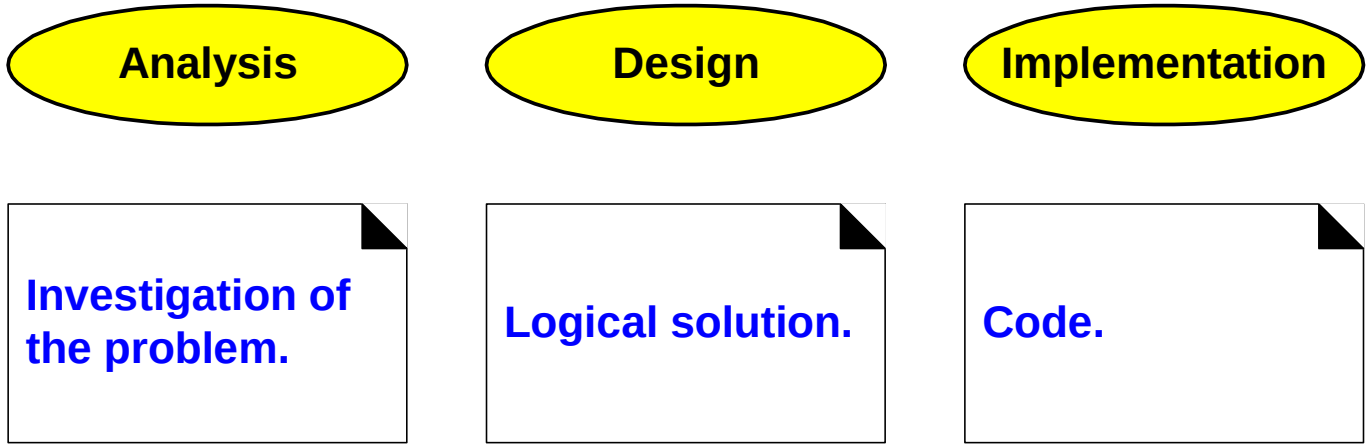
- Mapping Use Cases to Objects
- Modeling Associations and Aggregations
- Modeling Inheritance Relationships

4.5 Reviewing Analysis and Design Models

- Iterative Refinement
- Client Sign-Off Procedures

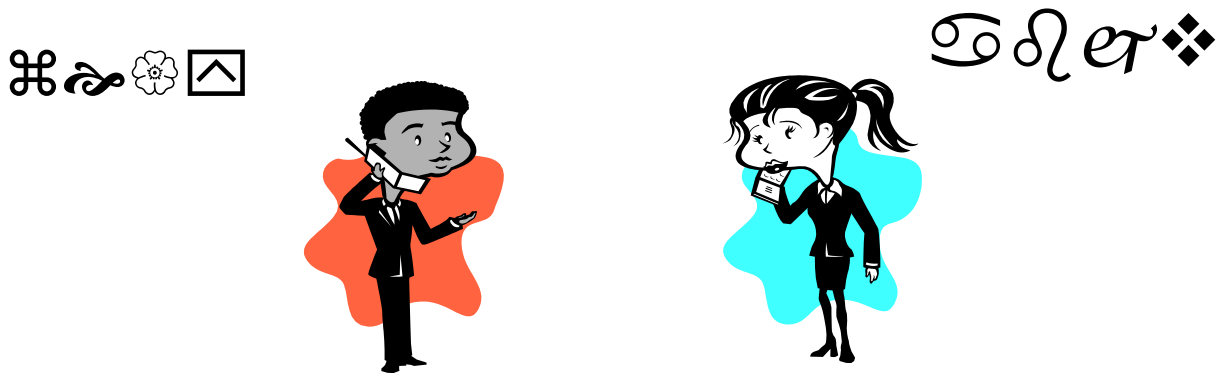
Object Orientation

- Object-oriented analysis.
  - Investigation that is object-centric.
- Object-oriented design.
  - Solution in terms of interacting software objects.
- Object-oriented programming.
  - Coding in an object-oriented programming language.



Why Object-Oriented?

*"The "software crises" came about when people realized the major problems in software development were … caused by **communication** difficulties and the management of **complexity"** [Budd]*



What kind of language can alleviate difficulties with communication & complexity hopefully well?

Why Object-Oriented?

- Communication & complexity about the problem and the solution, all expressed in terms of ***concepts*** in a language!
- But then, What is CONCEPT? [Martin & Odell]
- *Consider Human Growth & Concept Formation*

stage	concepts
infant	*the world is a buzzing confusion*
very young age	*"blue" "sky" (individual concepts)* *"blue sky" (more complex concept)* *hypothesis: humans possess an innate capacity for perception*
getting older	*-> increased meaning, precision, subtlety, ...* *the sky is blue only on cloudless days* *the sky is not really blue* *it only looks blue from our planet Earth* *because of atmospheric effects* *elaborate conceptual constructs*

Concept formation: from chaos to order!

Why Object-Oriented?

concepts and objects

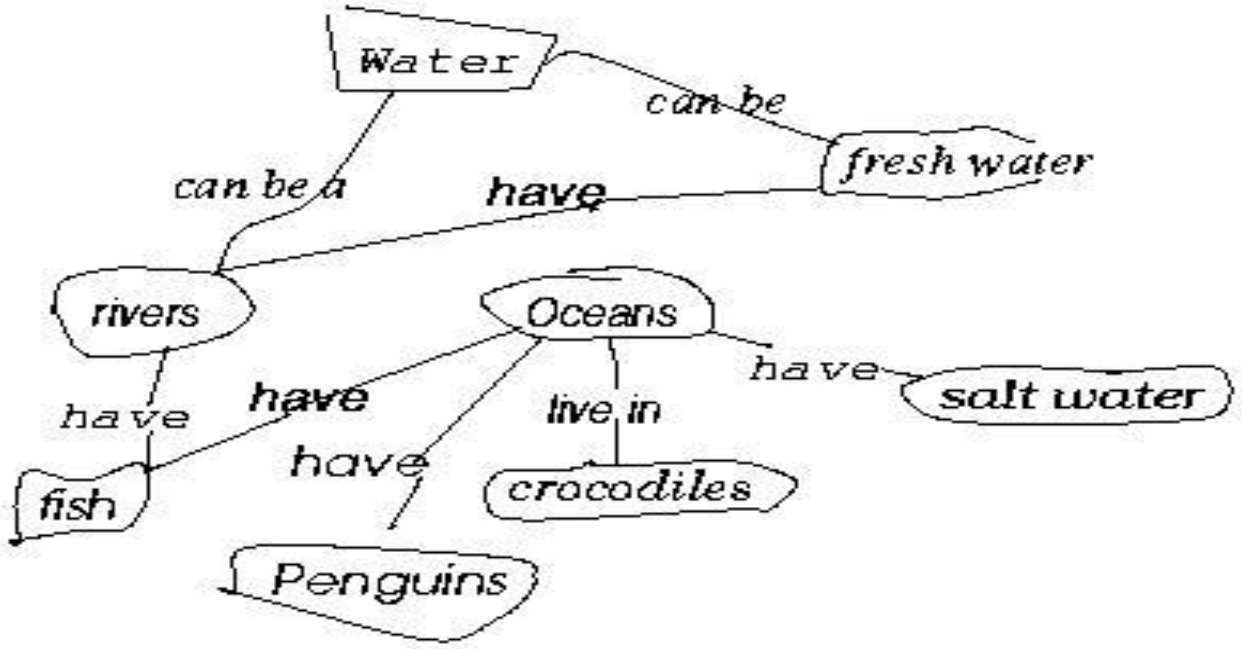
So, concepts are needed to bring order … into

But, What is CONCEPT? [Martin & Odell] [Nova 1984, Cambridge University Press]

Study of a first grade class

When given a list of concepts (water, salt water, Oceans, Penguins,...),

Harry constructed a concept diagram through whic understands his world and communicates meaning

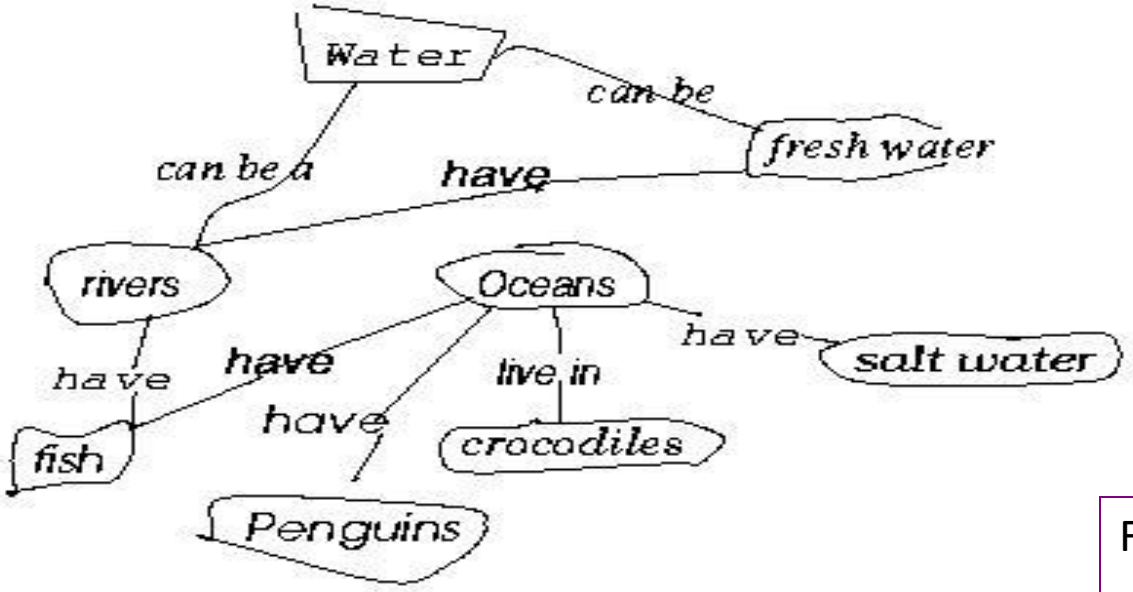


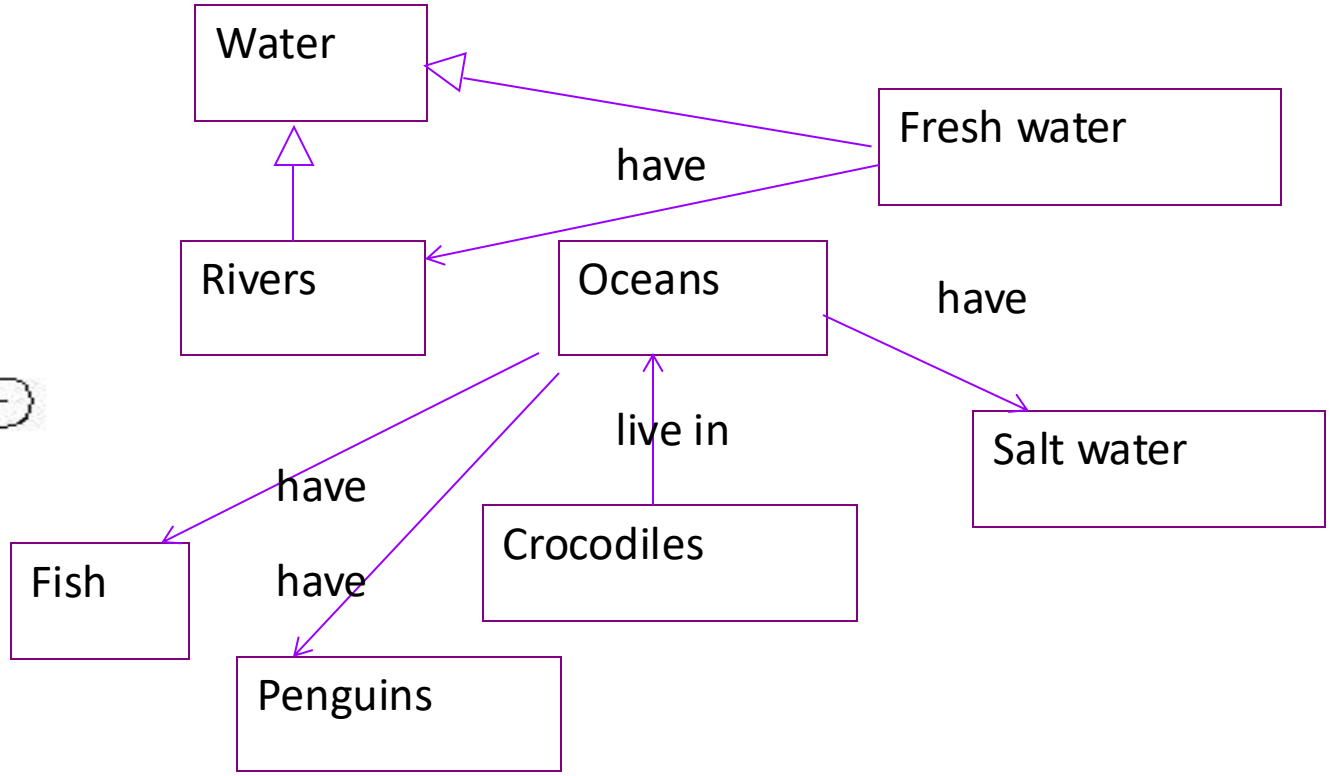
Does Harry understand the concepts? Do you understand what Harry understands? Agree or Diagree?

Why Object-Oriented?

for Conceptual … Modeling Reasons

What kind of language can be used to create this concept diagram, or Harry's mental image?





Why Object-Oriented?

What is a ***model* and why**?

- A model is a simplification of reality.

  E.g., a miniature bridge for a real bridge to be built

  - *Well…sort of….but not quite*
  - A model is *our* simplification of *our perception* of reality

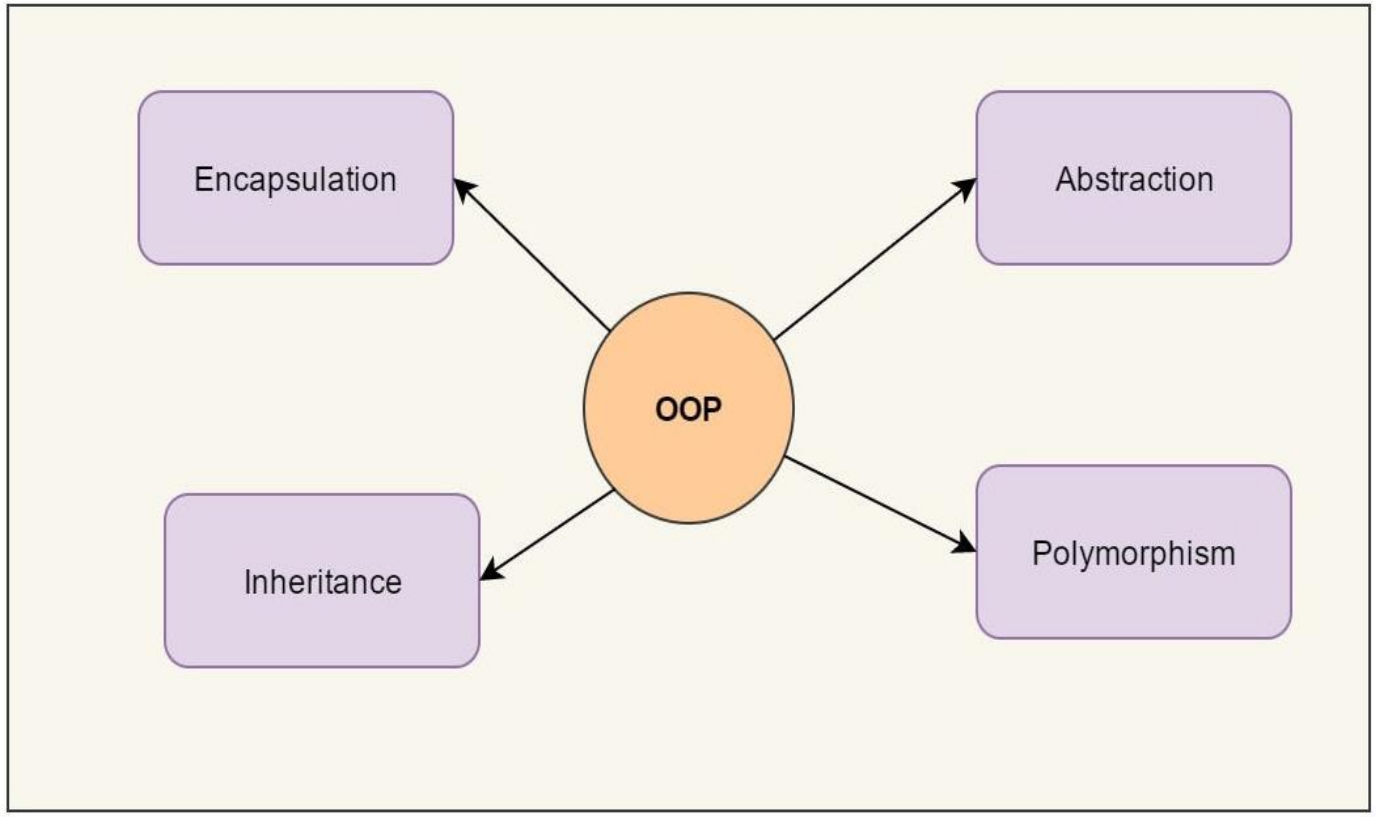
    (that is, if it exists, otherwise it could be a mere illusion).

    communication is not about reality but about your/my/his/her perception of reality => validation and verification hard but needed

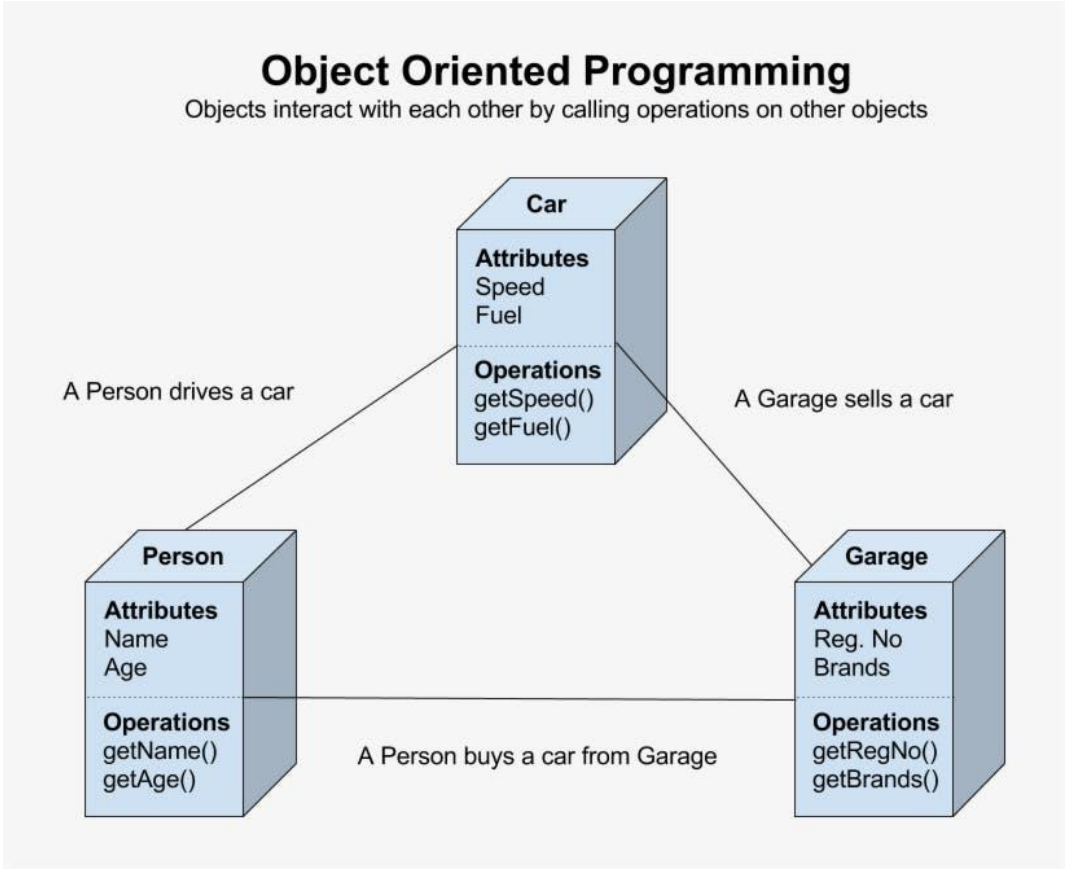
- A model is an *abstraction* of something for the purpose of *understanding*, be it the problem or a solution.

- To understand *why* a software system is needed, *what* it should do, and *how* it should do it.
- To communicate our understanding of why, what and how.
- To detect commonalities and differences in your perception, my perception, his perception and her perception of reality.
- To detect misunderstandings and miscommunications.

OOAD Principles

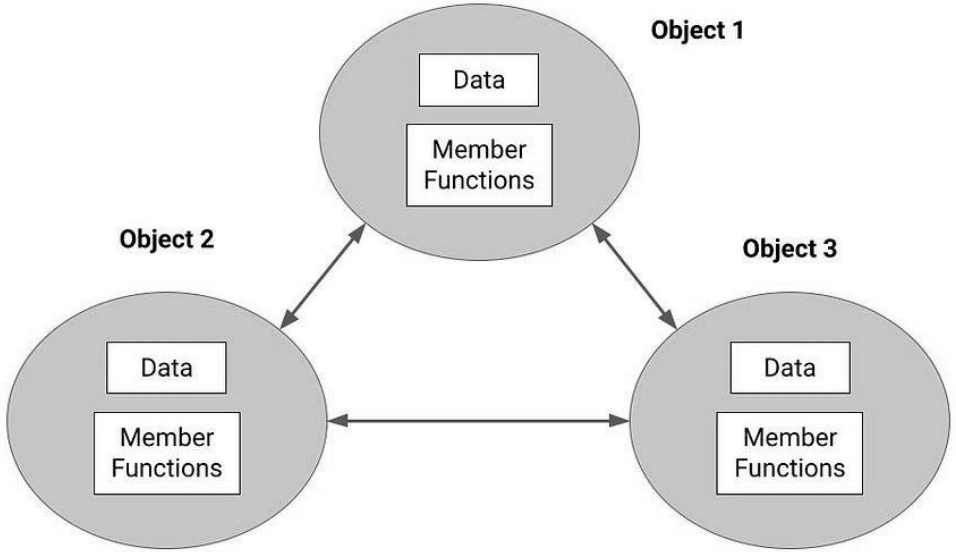


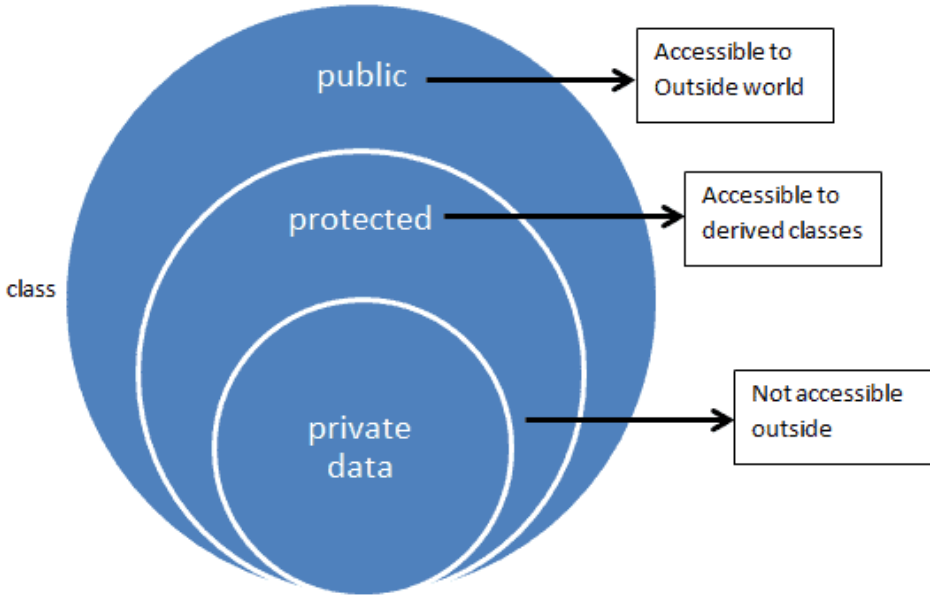
Four Pillars of Object Oriented Programming



OOAD Principles

- **Encapsulation**
  - Mechanism by which we combine ***data*** and the ***functions*** that manipulate the data into one unit
  - Objects & Classes enforce encapsulation





OOAD Principles

- **Abstraction**
  - Abstraction, in the context of OOP, refers to the ability to hide complex implementation details and show only the necessary features of an object.
  - This simplifies the interaction with objects, making programming more intuitive and efficient.
  - It provides a clear separation between *what an object does* and *how it achieves its functionality*, fostering a higher level of understanding and collaboration among developers.
  - You just need to know which methods of the object are available to call and which input parameters are needed to trigger a specific operation.
  - But you don't need to understand how this method is implemented and which kinds of actions it has to perform to create the expected result.

OOAD Principles

- **Abstraction**

Data Abstraction and process Abstraction??
Abstraction Vs Encapsulation??

Abstraction

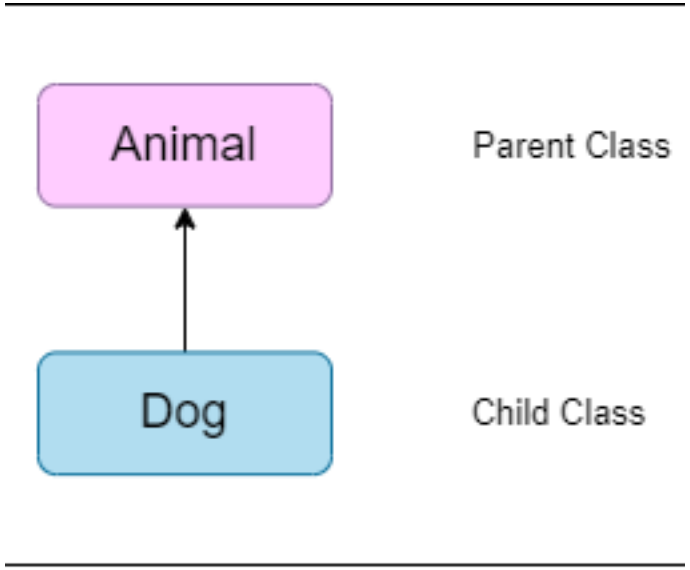
Focus on the essential
Omits tremendous amount of details
…Focus on what an object "is and does"

Encapsulation

a.k.a. information hiding
Objects encapsulate:
property
behavior as a collection of methods invoked by messages
…state as a collection of instance variables

OOAD principles

- **Inheritance**
  - Inheritance is a mechanism that allows us to create hierarchies of classes, facilitating the sharing of properties and methods among them.
  - Inheritance in oops is based on a hierarchical relationship between classes, where a derived class (also known as a subclass or child class) inherits the characteristics of a base class (also known as a superclass or parent class).
- The derived class extends the functionality of the base class by adding new features or overriding existing ones.



OOAD principles

- **Inheritance**
  - Inheritance facilitates the ***reuse of code*** by allowing a subclass to inherit attributes and methods from its superclass.
  - By inheriting features from a base class, modifications and updates can be made at the superclass level, affecting all derived classes.
  - This ensures consistency and simplifies maintenance tasks, as changes are centralized and propagate through the inheritance hierarchy.

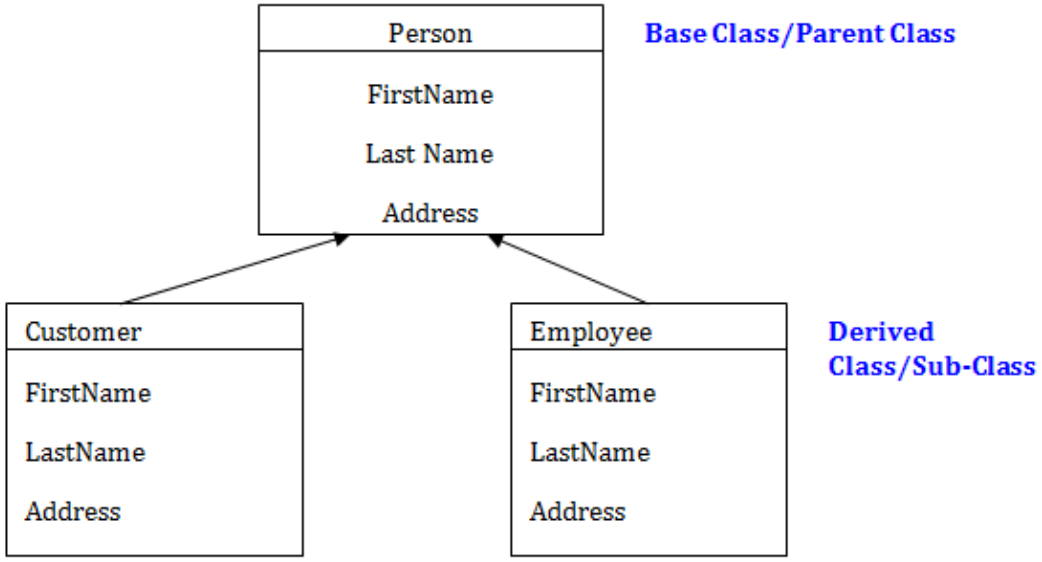
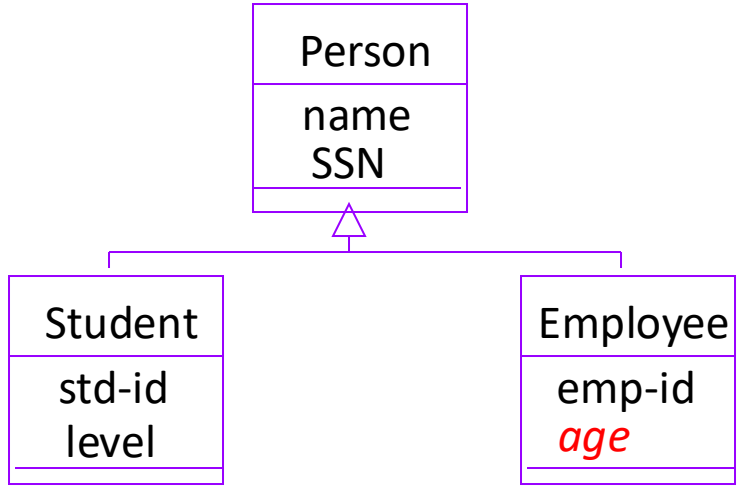


FIGURE 2: INHERITANCE

OOAD principles

•Inheritance

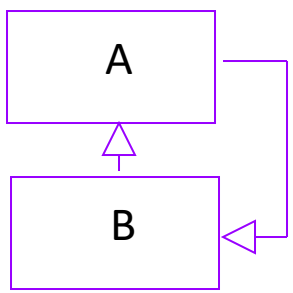
- Specialization: The act of defining one class as a refinement of another.
- Subclass: A class defined in terms of a specialization of a superclass using inheritance.
- Superclass: A class serving as a base for inheritance in a class hierarchy
- Inheritance: Automatic duplication of superclass attribute and behavior definitions in subclass.

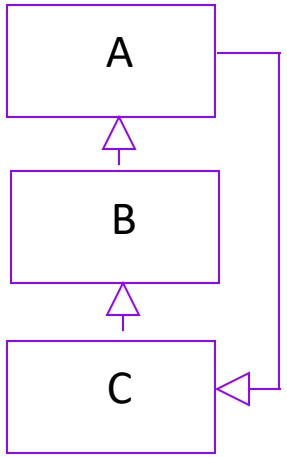


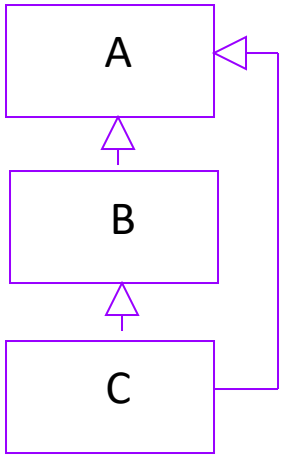
multiple inheritance?

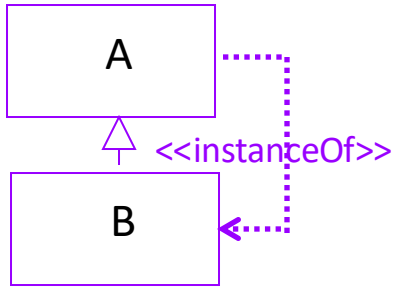
OOAD principles

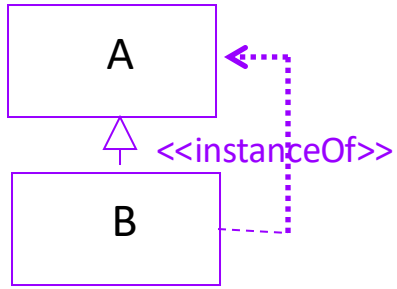
- **Inheritance**

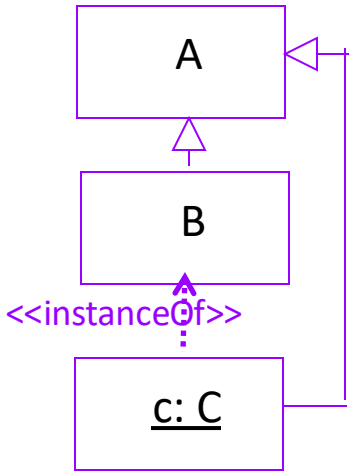


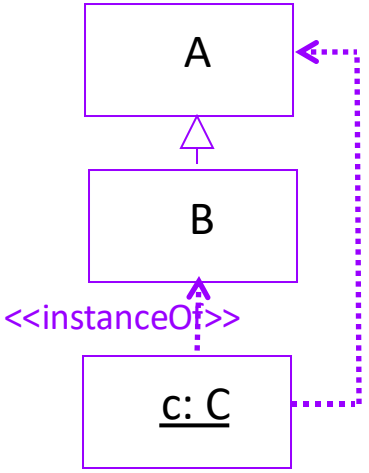






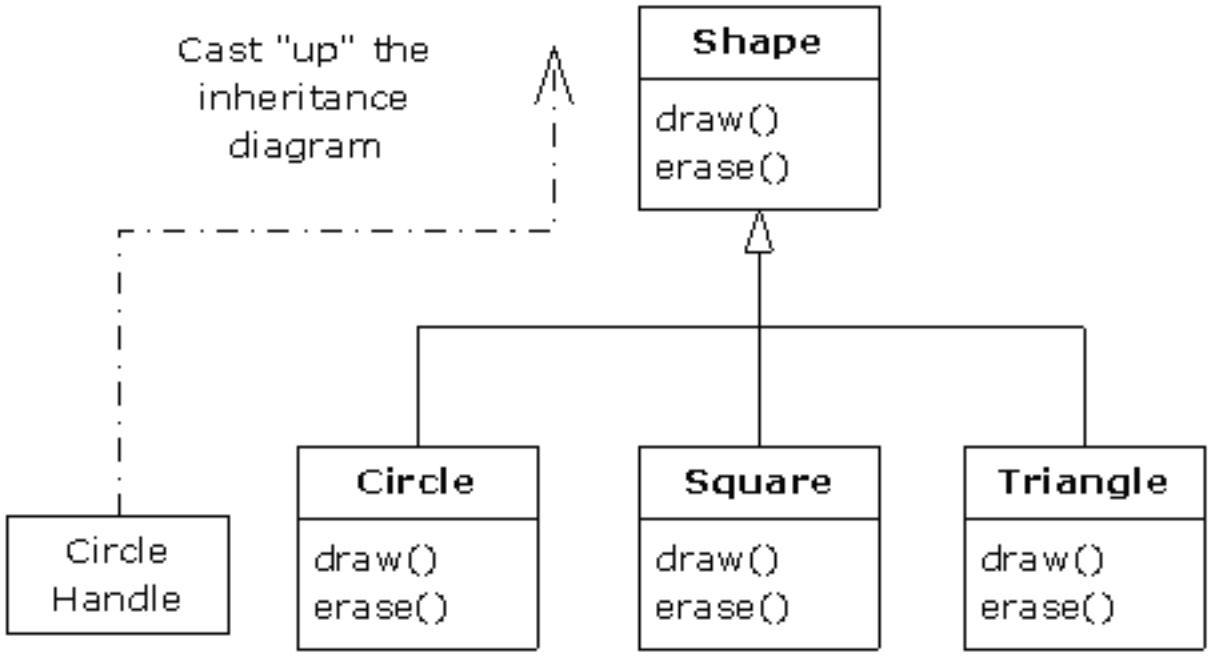






OOAD principles

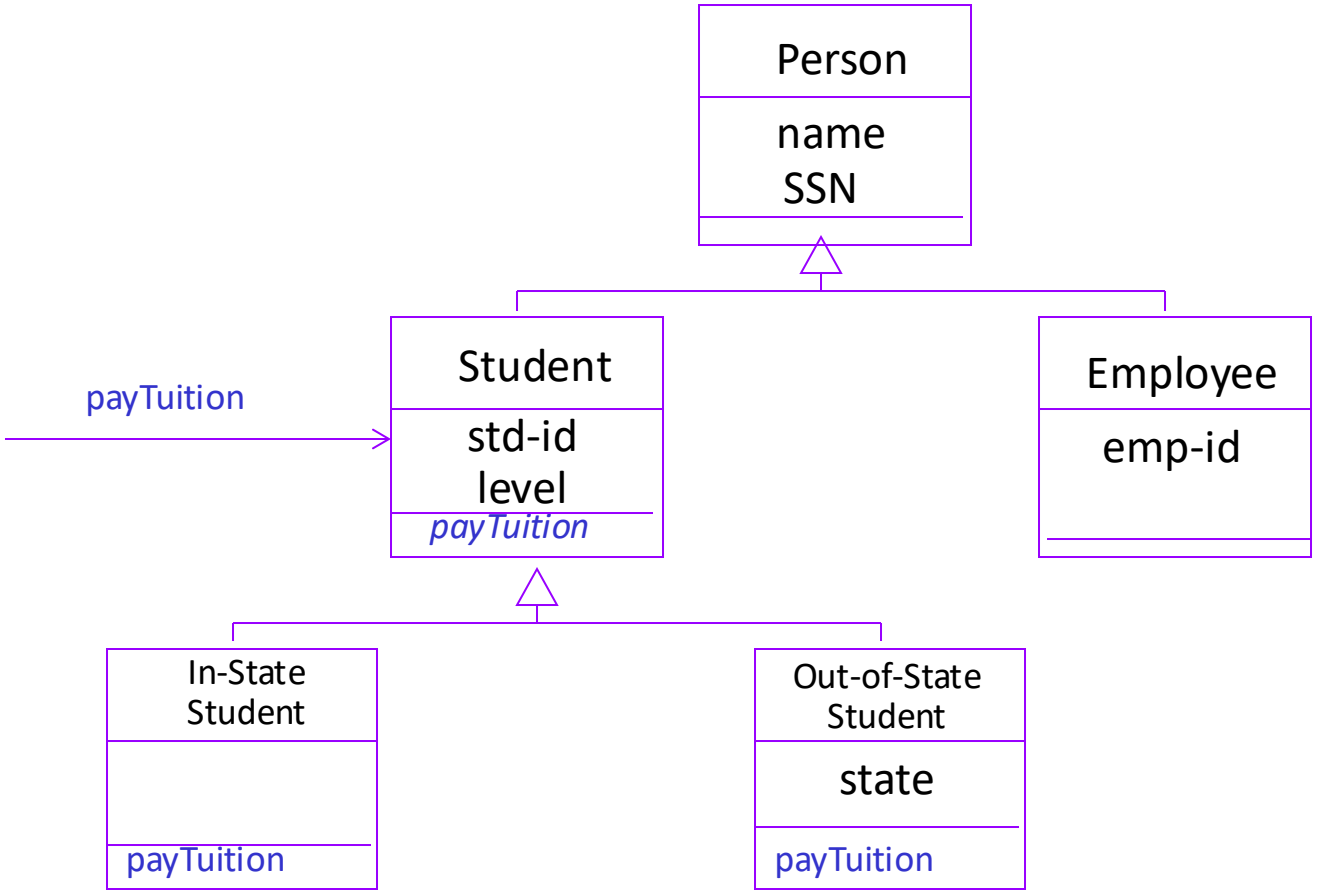
- **Polymorphism**
  - Polymorphism means many forms.
  - An entity that can exist in multiple forms brings polymorphism in object-oriented programming.
  - Concept of using different classes with the same interface. Each of these classes can provide its implementation of the interface.



OOAD principles

- **Polymorphism**

Objects of different classes respond to the same message differently.



OOAD principles

- **Polymorphism**

What is the advantage of polymorphism when
both *flm.fly()* and *j.fly()* give the same answer?

Polymorphism becomes helpful if you have multiple kinds of *FlyingMachine*. Then you could have a method that accepts any kind of *FlyingMachine* and uses its `fly()` method. An example might be `testMaxAltitude(FlyingMachine)`.

Example taken from: https://stackoverflow.com/questions/11064409/why-to-use-polymorphism

```
class FlyingMachines {
    public void fly() {
        System.out.println("No implementation");
    }
}

class Jet extends FlyingMachines {
    public void fly() {
        System.out.println("Start, Taxi, Fly");
    }

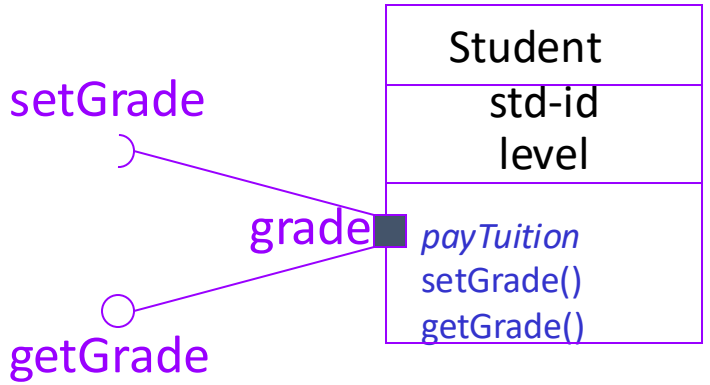
    public void bombardment() {
        System.out.println("Throw Missile");
    }
}

public class PolymorphicTest {
    public static void main(String[] args) {
        FlyingMachines flm = new Jet();
        flm.fly();

        Jet j = new Jet();
        j.bombardment();
        j.fly();
    }
}
```

OOAD principles

- **Interfaces**
  - Information hiding - all data should be hidden within a class, at least in principle.
  - make all data attributes private
  - provide public methods to get and set the data values (cf. Java design patterns)
  - e.g. Grade information is usually confidential, hence it should be kept private to the student. Access to the grade information should be done through *interfaces*, such as setGrade and getGrade



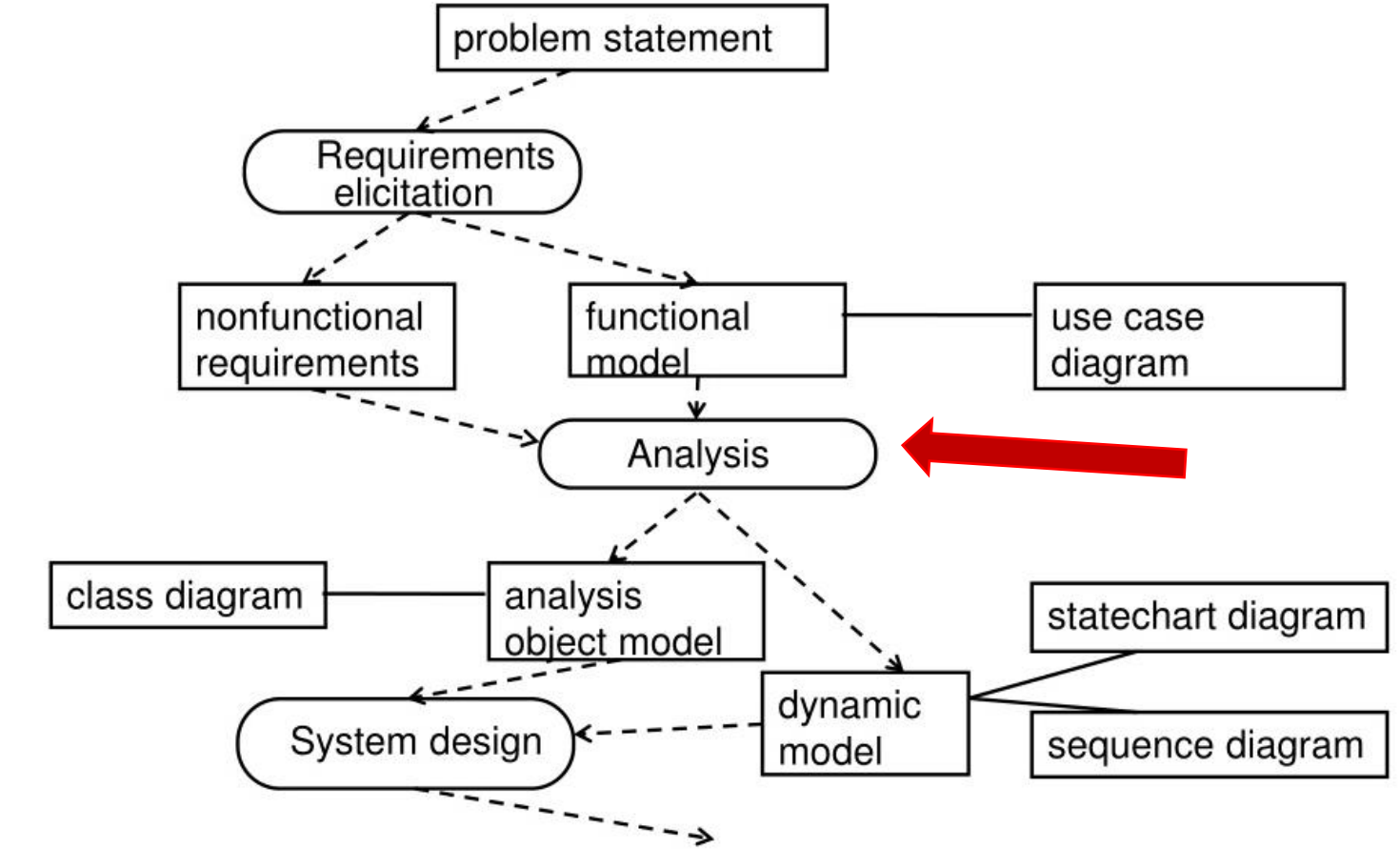
OOAD principles

- Abstract Class.
  - An *incomplete* superclass that defines common parts.
  - Not instantiated.
- Concrete class.
  - Is a *complete* class.
  - Describes a concept completely.
  - Is intended to be instantiated.

Analysis Object Models and Dynamic Models

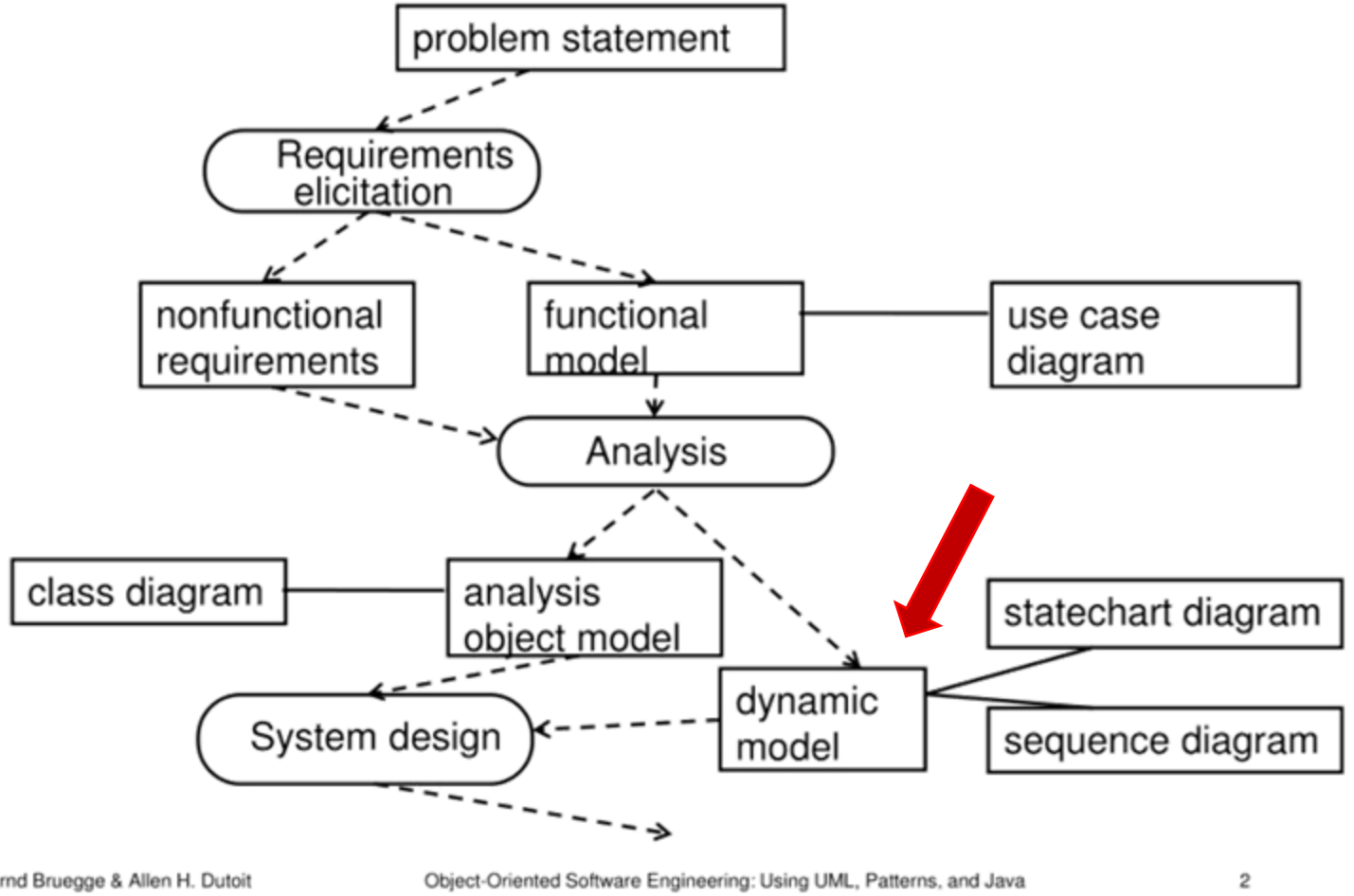
Analysis Object Models and Dynamic Models

- The ***analysis object model*** is a part of the analysis model and focuses on the individual concepts that are manipulated by the system, their properties and their relationships.
- The analysis object model, depicted with UML class diagrams, includes classes, attributes, and operations.
- The analysis object model is a visual dictionary of the main concepts visible to the user.



Analysis Object Models and Dynamic Models

- The ***dynamic model*** focuses on the behavior of the system.
- The dynamic model is depicted with sequence diagrams and with state machines.
- The dynamic model serves to assign responsibilities to individual classes and, in the process, to identify new classes, associations, and attributes to be added to the analysis object model.



Analysis Object Models and Dynamic Models

- When working with either the analysis object model or the dynamic model, it is essential to remember that these models represent ***user-level concepts***, not actual software classes or Components.

- ***Analysis classes*** should be viewed as high-level abstractions that will be realized in much more detail later.

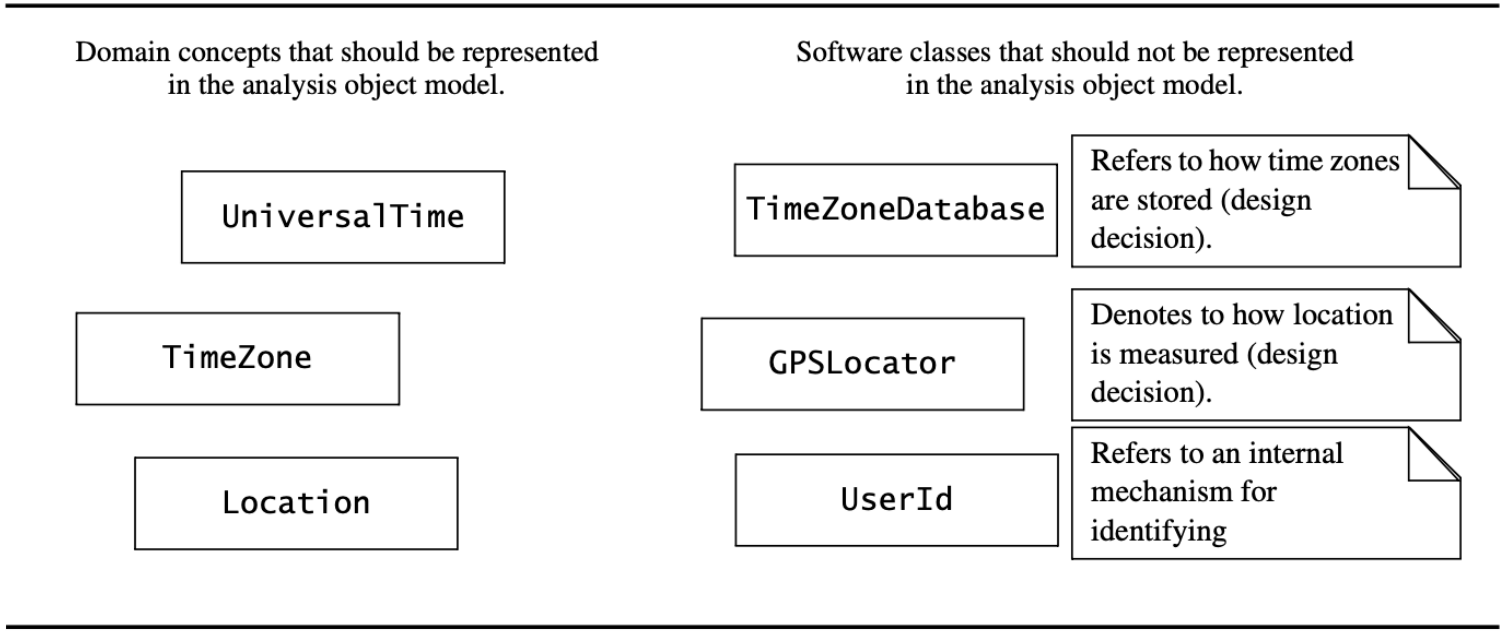
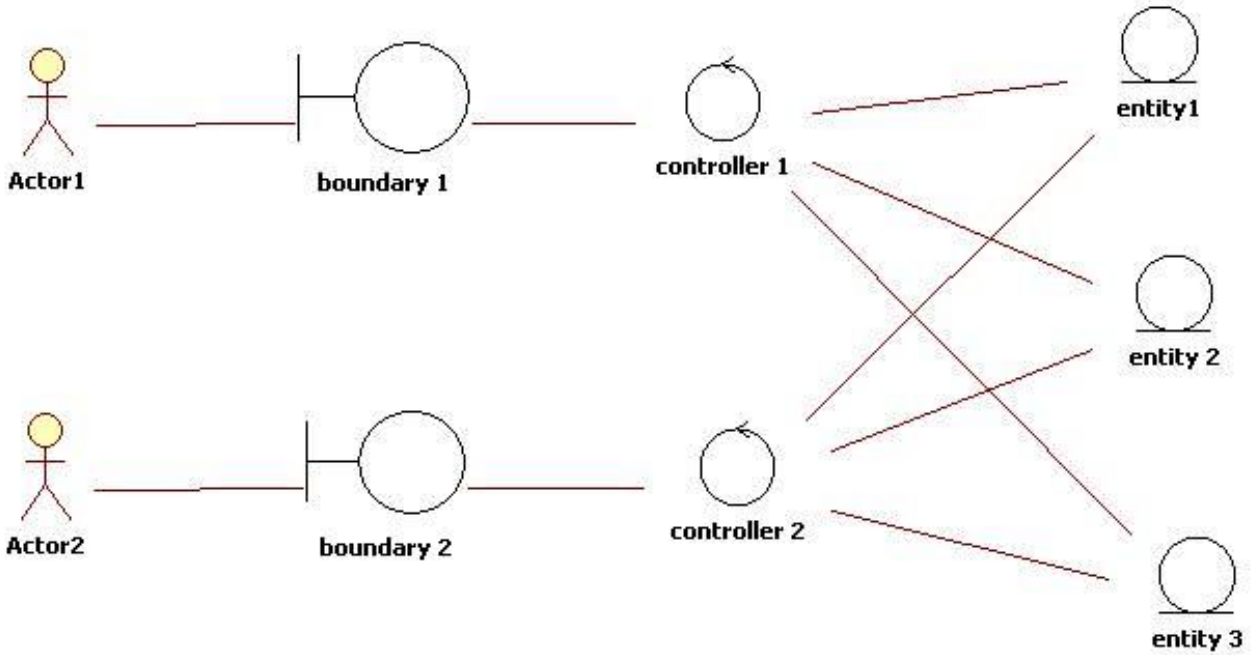


Figure 5-4 Examples and counterexamples of classes in the analysis object model of `SatWatch`.

Entity, Boundary, and Control Objects

- The analysis object model consists of entity, boundary, and control objects [Jacobson et al., 1999].
- ***Entity objects*** represent the persistent information tracked by the system.
- ***Boundary objects*** represent the interactions between the actors and the system.
- ***Control objects*** are in charge of realizing use cases.



Entity, Boundary, and Control Objects

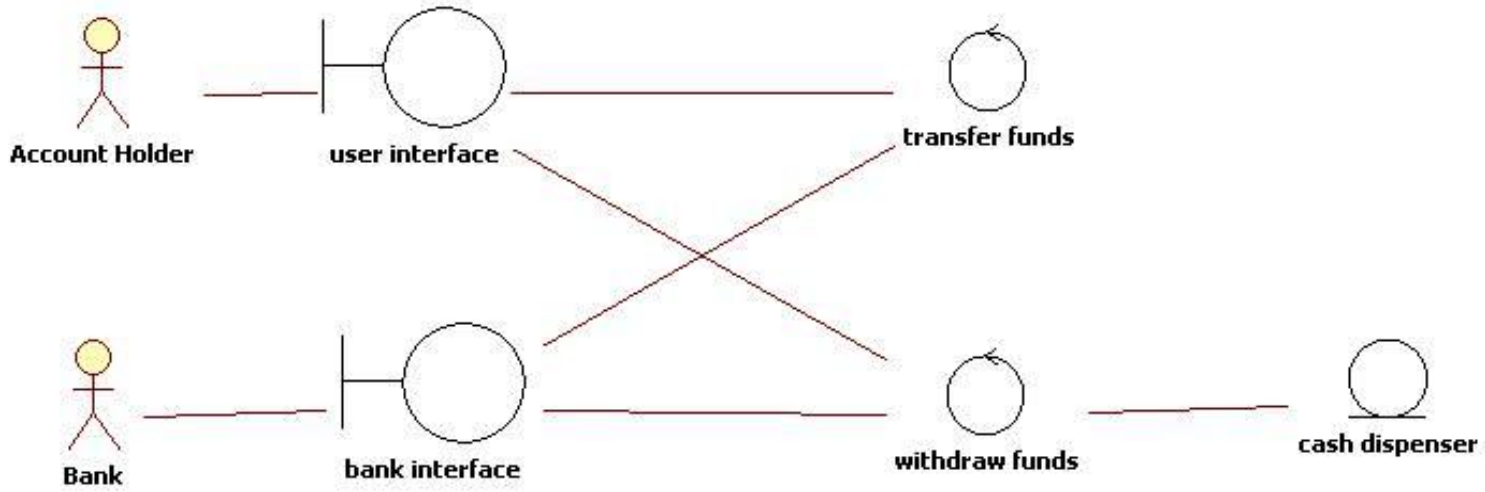
- Modeling the system with ***entity***, ***boundary***, and ***control objects*** provides developers with simple heuristics to distinguish different, but related concepts.
- The ***three-object-type*** approach also leads to models that are more resilient to change: the interface to the system (represented by the boundary objects) is more likely to change than its basic functionality (represented by the entity and control objects).
- By separating the ***interface*** from the ***basic functionality***, we are able to keep most of a model untouched when, for example, the user interface changes, but the entity objects do not.



Figure 5-5 Analysis classes for the `2Bwatch` example.

Entity, Boundary, and Control Objects

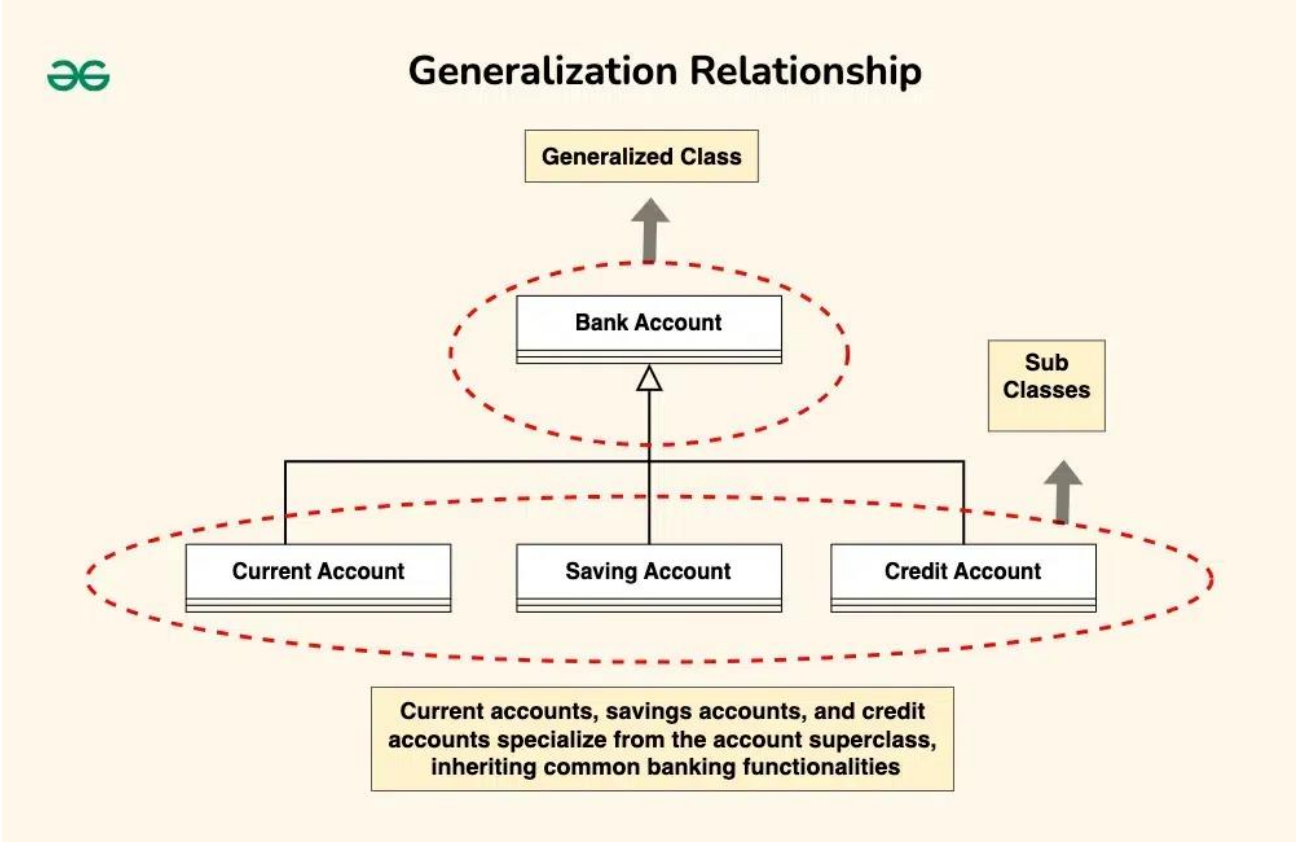
ATM Example



- ***Entities*** are objects representing system data: Customer, Transaction, Cart, etc.
- ***Boundaries*** are objects that interface with system actors: user interfaces, gateways, proxies, etc.
- ***Controllers*** are objects that mediate between boundaries and entities. They orchestrate the execution of commands coming from the boundary.

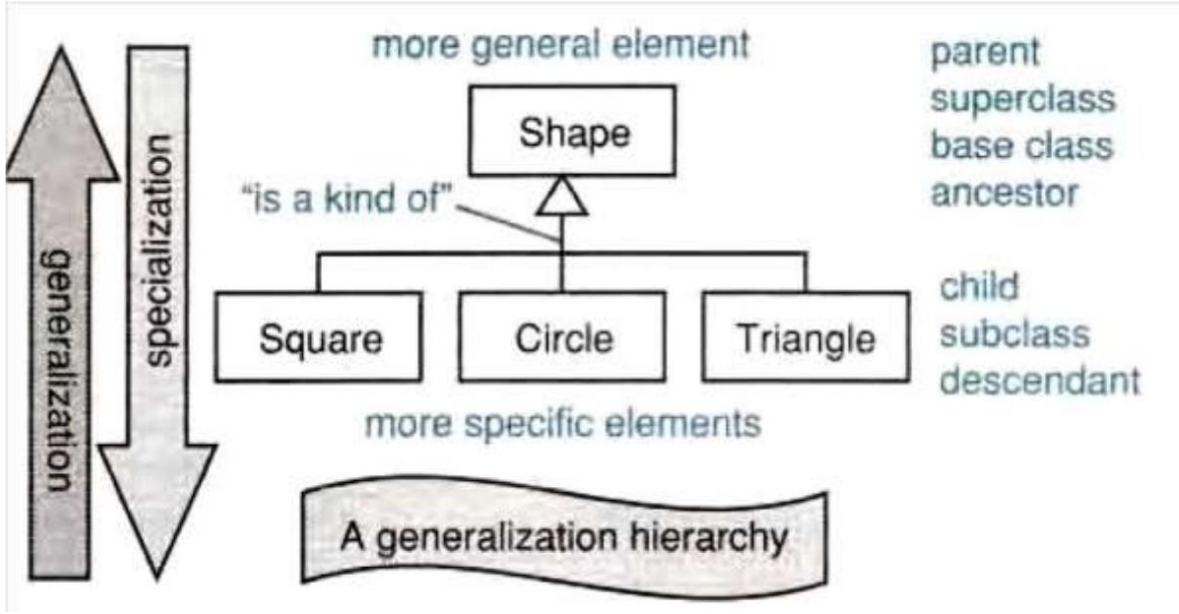
Generalization and Specialization

- ***Generalization*** is the modeling activity that identifies abstract concepts from lower-level ones.
- Generalization is a relationship between a more general thing and a more specific thing:
  - The more specific thing is consistent in every way with the more general thing.
  - The ***substitutability*** principle states that you can substitute the more specific thing anywhere the more general thing is expected.
  - Generalization applies to all classifiers and some other modeling elements.
  - It embodies the "***is-a***" relationship, where a subclass is said to be a specialized version of its superclass, inheriting its attributes and behaviors while potentially adding or modifying its own.



Generalization and Specialization

- ***Specialization*** is the activity that identifies more specific concepts from a high-level one. For example, assume that we are building an emergency management system from scratch and that we are discussing its functionality with the client.
- It represents an "***is-a-kind-of***" relationship, where subclasses are more specialized versions of the superclass.



Generalization hierarchies may be created by generalizing from specific things or by specializing from general things.

Generalization and Specialization

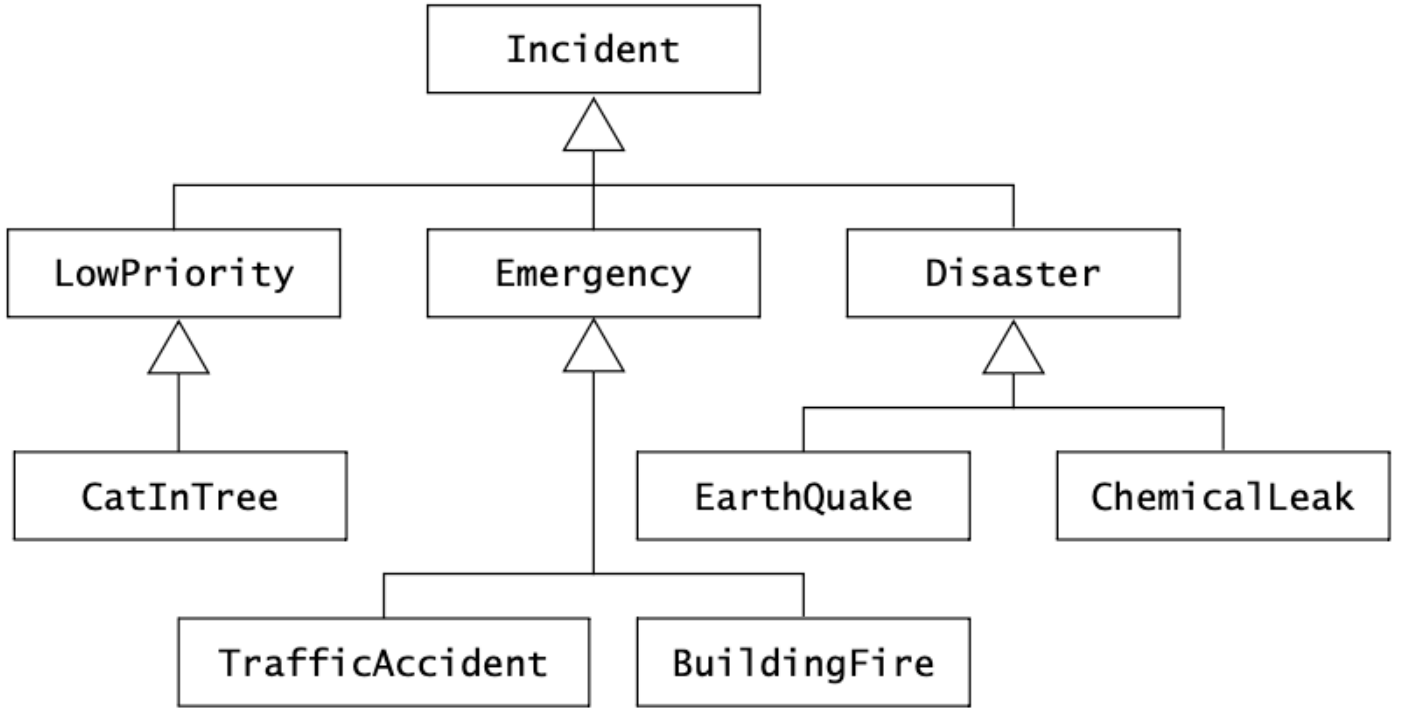


Figure 5-6 An example of a generalization hierarchy (UML class diagram). The top of the hierarchy represents the most general concept, whereas the bottom nodes represent the most specialized concepts.